

Revisiting Honest Re-Encryption Attack for Proxy Re-Encryption Schemes

Haotian Yin¹, Jie Zhang¹, Yuji Dong², Dominik Wojtczak³, Eng Gee Lim¹

¹ School of Advanced Technology,
Xi'an Jiaotong-Liverpool University, Suzhou, 215123, China
`haotian.yin23@student.xjtlu.edu.cn, {jie.zhang01, enggee.lim}@xjtlu.edu.cn`

² School of Internet of Things,
Xi'an Jiaotong-Liverpool University, Suzhou, 215400, China
`yuji.dong02@xjtlu.edu.cn`

³ Department of Computer Science,
University of Liverpool, Liverpool, L69 3BX, UK
`d.wojtczak@liverpool.ac.uk`

Abstract. Proxy re-encryption (PRE) schemes allow a proxy to transform ciphertexts for a delegatee. While the notion of indistinguishability under honest re-encryption attacks (IND-HRA) has become a standard for lattice-based PREs, its relationship with IND-CCA remains nuanced. In this work, we prove that IND-CCA implies IND-HRA for single-hop PRE schemes. For multi-hop and probabilistic re-encryption, we prove that a slightly modified IND-CCA implies IND-HRA by pre-querying. By generalising source-hiding (SH), we further demonstrate that IND-CPA, combined with a strictly weaker SH variant, is sufficient for IND-HRA. Additionally, we introduce re-encryption key unlearnability (RKUL), a property capturing the hardness of recovering re-encryption keys from ciphertexts. We prove that RKUL is implied by both IND-CCA and IND-HRA. Finally, we revisit several existing schemes claimed to be IND-HRA secure and demonstrate that they fail to satisfy RKUL, thus invalidating their IND-HRA security claims.

Keywords: Honest re-encryption attack · Proxy re-encryption · Chosen ciphertext attack

1 Introduction

Proxy re-encryption (PRE) schemes allow a proxy to transform a ciphertext intended for a delegator into one decryptable by a delegatee, without revealing the underlying plaintext. PRE schemes are classified by directionality (unidirectional vs. bidirectional) and transformability (single-hop vs. multi-hop). This work focuses on unidirectional schemes, where re-encryption keys are generated non-interactively by the delegator.

1.1 Security Models: CPA, HRA, and CCA

Security for PRE is typically defined via indistinguishability games.

- IND-CPA [1]: The adversary has access to re-encryption key generation but cannot decrypt or ask for re-encryptions.
- IND-HRA [4]: Introduced to capture “honest re-encryption attacks” by malicious delegates. It adds a re-encryption oracle to the CPA setting, preventing the adversary from trivially decrypting valid re-encryptions of the challenge ciphertext.
- IND-CCA [2, 9]: The strongest notion, granting access to decryption and re-encryption oracles.

While IND-CCA implies IND-CPA, its relationship with IND-HRA is subtle. Cohen [4] suggested they might be incompatible because the IND-CCA challenger, lacking the ability to identify all probabilistic re-encryptions ([8] showed that IND-CCA implies IND-HRA for deterministic re-encryptions) of a challenge ciphertext, must conservatively reject legitimate queries to prevent trivial wins. Consequently, current proofs relying on re-encryption simulatability (RES) or source-hiding (SH) [7] typically detour through IND-CPA to achieve IND-HRA, bypassing the powerful IND-CCA guarantees.

1.2 Our Contributions

In this work, we bridge the gap between IND-CCA and IND-HRA, critically evaluating the security of existing schemes. Our contributions are fourfold:

Reducing IND-CCA to IND-HRA. We demonstrate that IND-CCA is strictly stronger than IND-HRA with some minor adjustments to the IND-CCA challenger.

- Single-Hop PRE: We show that for single-hop schemes, IND-CCA directly implies IND-HRA. Since genuine derivatives of the challenge cannot be re-encrypted further (due to the hop limit), the IND-CCA challenger’s tracking becomes precise, allowing a perfect simulation of the IND-HRA game.
- General PRE: We demonstrate that IND-CCA’ (defined in Section 4) implies IND-HRA without any additional assumptions. We leverage the upper bound of the ciphertext that an adversary might query and a more compact derivation definition.

Weaker Assumption for achieving IND-HRA. A common approach to constructing an IND-HRA-secure scheme is to first construct an IND-CPA-secure scheme and then prove it is SH to complete the process.

- Generalised Source-Hiding: We generalise the notion of Source-Hiding, denoted as $\text{SH}^{(a,b)}$, where an a -hop ciphertext is indistinguishable from a b -hop one. We prove that a specific variant, $\text{SH}^{(a,a)}$, combined with IND-CPA, suffices to imply IND-HRA. Crucially, this notion is strictly weaker than the standard $\text{SH}^{(0,1)}$ (fresh ciphertext is indistinguishable from a re-encrypted ciphertext).

Re-Encryption Key Unlearnability (RKUL). We introduce a novel necessary condition for PRE security called *Re-Encryption Key Unlearnability* (RKUL). RKUL captures the adversary’s inability to recover the re-encryption key given input-output ciphertext pairs.

- We prove that both IND-HRA and IND-CCA imply RKUL, establishing it as a fundamental requirement.
- We demonstrate that RKUL is non-trivial and not implied by standard IND-CPA security.

Analysis of Existing Schemes. We use RKUL as a tool to audit existing lattice-based PRE schemes claimed to be IND-HRA secure.

- We identify a common design flaw where deterministic, linear re-encryption transformations leak the re-encryption key. By solving systems of linear equations derived from ciphertext pairs, an adversary can recover the key.
- We demonstrate that schemes such as [12] fail to satisfy RKUL and are thus not IND-HRA secure.

1.3 Roadmap

Section 2 introduces preliminaries. Section 3 proves the reduction from IND-CCA to IND-HRA for single-hop schemes. Section 4 proves the reduction from IND-CCA’ to IND-HRA. Section 5 details the generalised SH framework. Section 7 defines RKUL and its relationships with other notions. Finally, Section 8 presents attacks on existing schemes using RKUL.

2 Formal Definitions

Let λ be the security parameter and L the maximum re-encryption level. We use standard notations for PPT algorithms, negligible functions, and indistinguishability ($\approx_s$).

2.1 Proxy Re-Encryption

A unidirectional single-hop PRE scheme consists of algorithms $\text{Setup}(1^\lambda, 1^L) \rightarrow \text{pp}$, $\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$, $\text{ReKeyGen}(\text{pk}_i, \text{sk}_i, \text{pk}_j) \rightarrow \text{rk}_{i \rightarrow j}$, $\text{Enc}(\text{pk}, \text{m}, \ell) \rightarrow (\text{ct}, \ell)$, $\text{ReEnc}(\text{rk}_{i \rightarrow j}, \text{ct}_i, \ell) \rightarrow (\text{ct}_j, \ell + 1)$, and $\text{Dec}(\text{sk}, \text{ct}, \ell) \rightarrow \text{m} / \perp$. We omit the formal correctness definition as it follows the standard requirement that decryption recovers the message after valid encryption and at most one re-encryption (since $L = 2$ for single-hop).

2.2 Security Models

We focus on two security notions: IND-HRA [4] and IND-CCA [2]. To facilitate comparison and reduction, we present a unified game framework. The key difference lies in the permissiveness of the re-encryption oracle and the definition of “derivatives” (ciphertexts derived from the challenge).

Definition 1 (Security Game Framework). *The game proceeds between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ in three phases:*

Phase 1 (Setup & Keys): $\mathcal{C}$ generates pp . $\mathcal{A}$ queries key generation to generate honest keys (Hon) and corrupted keys (Cor).

Phase 2 (Oracles): $\mathcal{A}$ issues queries. $\mathcal{C}$ maintains sets ReKeys (generated re-encryption keys) and Deriv (derivatives of the challenge).

- $\mathcal{O}_{\text{ReKeyGen}}(\text{pk}_i, \text{pk}_j)$: Returns $\text{rk}_{i \rightarrow j}$. Rejects if $\text{pk}_i \in \text{Hon} \wedge \text{pk}_j \in \text{Cor}$ (preventing trivial exposure).
- $\mathcal{O}_{\text{Challenge}}(\text{pk}^*, \text{m}_0, \text{m}_1, \ell^*)$: (Once) Checks $\text{pk}^* \in \text{Hon}$. Samples $b \leftarrow \{0, 1\}$, returns $\text{ct}^* \leftarrow \text{Enc}(\text{pk}^*, \text{m}_b, \ell^*)$. Adds $(\text{pk}^*, \text{ct}^*, \ell^*)$ to Deriv .
- $\mathcal{O}_{\text{Enc}}(\text{pk}, \text{m}, \ell)$: (IND-HRA Only) Returns $\text{Enc}(\text{pk}, \text{m}, \ell)$ and tracks the ciphertext.
- $\mathcal{O}_{\text{ReEnc}}(\text{pk}_i, \text{pk}_j, \text{ct}_i, \ell)$: Returns $(\text{ct}_j, \ell + 1) \leftarrow \text{ReEnc}(\text{rk}_{i \rightarrow j}, \text{ct}_i, \ell)$. Rejects if $\text{pk}_j \in \text{Cor} \wedge (\text{ct}_i, \ell) \in \text{Deriv}$.

Phase 3 (Guess): $\mathcal{A}$ outputs b' . $\mathcal{A}$ wins if $b' = b$.

IND-HRA vs. IND-CCA Specifics.

- IND-HRA: Includes $\mathcal{O}_{\text{Enc}}$. The challenger *only* accepts re-encryption queries for ciphertexts generated by $\mathcal{O}_{\text{Enc}}$ or $\mathcal{O}_{\text{ReEnc}}$ (honest ciphertexts). Here, Deriv tracks the specific challenge ciphertext and its re-encrypted descendants.
- IND-CCA (Reconciled): Does not include $\mathcal{O}_{\text{Enc}}$. The challenger accepts *arbitrary* ciphertexts for re-encryption. Consequently, the definition of Deriv is broader to prevent offline attacks:
 1. *Online Re-encryption:* If input ciphertext (of re-encryption query) is in Deriv , output ciphertext is in Deriv .
 2. *Offline Computation:* If $\mathcal{A}$ knows $\text{rk}_{i \rightarrow j}$ (via $\mathcal{O}_{\text{ReKeyGen}}$), some ciphertext of i is in Deriv , and the (ct_j, ℓ) decrypts to m_0 or m_1 (implicit check), $(\text{pk}_j, \text{ct}_j, \ell)$ is in Deriv .

Remark 1. As per [11], we consider our IND-CCA model without a decryption oracle ($\text{CCA}_{0,2}$), as it suffices for our implication proofs.

3 IND-CCA Implies IND-HRA for Single-Hop PRE

In this section, we demonstrate that for single-hop PRE schemes, IND-CCA security directly implies IND-HRA security without requiring additional assumptions like source-hiding. We construct a reduction $\mathcal{S}$ that simulates the IND-HRA

game by simply acting as a transparent relay between the IND-HRA adversary $\mathcal{A}$ and the IND-CCA challenger, while filtering out trivially invalid queries (e.g., mismatched public keys).

The core difference between the IND-HRA and IND-CCA models lies in how they track and reject derivatives of the challenge ciphertext. We show that in the single-hop setting, these rejection criteria become functionally equivalent.

3.1 Oracle Discrepancies

Comparing the definitions of IND-CCA and IND-HRA, the only divergence appears in the re-encryption oracle, specifically regarding which input ciphertexts are considered “derivatives” of the challenge ciphertext (ct^*, ℓ^*) and thus rejected.

Let CCA.Deriv and HRA.Deriv be the sets of ciphertexts tracked as derivatives in the respective games. The IND-CCA challenger rejects any re-encryption query $(\text{pk}_i, \text{pk}_j, \text{ct}, \ell)$ if $(\text{pk}_i, \text{ct}, \ell) \in \text{CCA.Deriv}$, whereas the IND-HRA challenger rejects based on $(\text{ct}, \ell) \in \text{HRA.Deriv}$. While $\text{HRA.Deriv} \subseteq \text{CCA.Deriv}$ generally holds, the gap $\text{CCA.Deriv} \setminus \text{HRA.Deriv}$ represents derivatives generated via offline computation (e.g., re-encrypting the challenge locally) that the CCA challenger would reject but the HRA challenger might accept.

3.2 Equivalence in Single-Hop Setting

We now prove that this gap vanishes for single-hop PRE schemes.

Theorem 3.1 (Reduction for Single-Hop PRE). *Let PRE be a single-hop PRE scheme where re-encryption is only permitted for ciphertexts at level $\ell = 1$ (fresh) and outputs ciphertexts at level $\ell = 2$ (re-encrypted). If PRE is IND-CCA secure, then it is also IND-HRA secure.*

Proof. Consider a reduction $\mathcal{S}$ that forwards all valid queries from the IND-HRA adversary $\mathcal{A}$ to the IND-CCA challenger. We analyse the acceptance of a re-encryption query on an input ciphertext (ct, ℓ) .

In a single-hop setting, any valid input to the re-encryption oracle must be at level $\ell = 1$.

- **Case 1:** Input is the challenge itself. If $(\text{ct}, \ell) = (\text{ct}^*, \ell^*)$, both challengers reject the query (in IND-CCA as a direct challenge access, in IND-HRA as a trivial attack).
- **Case 2:** Input is an offline derivative. If $\mathcal{A}$ locally re-encrypts (ct^*, ℓ^*) to obtain a derivative $(\text{ct}', \ell^* + 1)$, this new ciphertext is at level $\ell = 2$. Since the scheme is single-hop, any subsequent re-encryption query using $(\text{ct}', \ell^* + 1)$ is invalid and will be rejected by *both* challengers regardless of their derivative tracking rules.

Consequently, the set of re-encryptable derivatives is limited to the challenge ciphertext itself. The condition that distinguishes IND-CCA from IND-HRA (tracking offline re-encrypted derivatives) becomes redundant because such derivatives cannot be re-encrypted again. Thus, $\mathcal{S}$ provides a perfect simulation.

□

4 IND-CCA' Implies IND-HRA

4.1 Tighter IND-CCA

To facilitate the reduction, we refine the standard IND-CCA model into a variant we denote as IND-CCA'. In this variant, the challenger's re-encryption oracle tracks all processed ciphertexts, behaving analogously to the encryption oracle in the IND-HRA game. Specifically, the challenger maintains a set Ct containing all valid ciphertexts it has generated or received as valid inputs:

- $\mathcal{O}_{\text{ReEnc}}(\text{pk}_i, \text{pk}_j, \text{ct}_i, \ell)$: Returns $(\text{ct}_j, \ell + 1) \leftarrow \text{ReEnc}(\text{rk}_{i \rightarrow j}, \text{ct}_i, \ell)$. The oracle rejects the query if $\text{pk}_j \in \text{Cor} \wedge (\text{ct}_i, \ell) \in \text{Deriv}$. Upon success, it adds both input $(\text{pk}_i, \text{ct}_i, \ell)$ and output $(\text{pk}_j, \text{ct}_j, \ell + 1)$ to Ct .

Crucially, we relax the rejection condition for derivatives generated via offline computation. The challenger only treats a ciphertext as a derivative if it has *not* been previously recorded in Ct . The modified rule is as follows:

- *Modified Offline Computation*: If $\mathcal{A}$ knows $\text{rk}_{i \rightarrow j}$ (via $\mathcal{O}_{\text{ReKeyGen}}$), some ciphertext of i is in Deriv' , and the (ct_j, ℓ) decrypts to m_0 or m_1 (implicit check), $(\text{pk}_j, \text{ct}_j, \ell)$ is in Deriv' if and only if:

$$\boxed{(\text{pk}_j, \text{ct}_j, \ell) \notin \text{Ct}}.$$

Let Deriv' denote the derivative set under this modified rule. We argue that this modification does not compromise the security of the challenge ciphertext, as it does not exclude any *genuine* derivatives:

1. Online Derivatives: If a genuine derivative $(\text{pk}_j, \text{ct}_j, \ell)$ is present in Ct , it must have been the output of an online query $\mathcal{O}_{\text{ReEnc}}(\dots)$. Consequently, its predecessor would be in Deriv' , and $(\text{pk}_j, \text{ct}_j, \ell)$ would naturally be included in Deriv' via the recursive definition of online derivatives.
2. Offline Derivatives: If a genuine derivative $(\text{pk}_j, \text{ct}_j, \ell)$ is *not* in Ct , it is captured precisely by the modified offline computation rule.

While the IND-CCA' challenger may still classify some independent ciphertexts as derivatives (conservative rejection), the scope of such “speculative” derivatives is narrowed. This allows the simulator to “whitelist” independent ciphertexts by ensuring they are included in Ct .

4.2 Reduction

Theorem 4.1. *If a PRE scheme is IND-CCA' secure, then it is also IND-HRA secure.*

Proof (Sketch). The core strategy of the reduction is *pre-filling*. The simulator $\mathcal{S}$ generates a sufficiently large pool of ciphertexts for the potential challenge messages $(\mathbf{m}_0, \mathbf{m}_1)$ and registers them with the IND-CCA' challenger. This ensures that these ciphertexts are included in Ct , thereby exempting them from the offline derivative check.

$\mathcal{S}$ acts as the IND-HRA challenger for $\mathcal{A}$. Let q_u be the total number of users and q_r be the upper bound on re-encryption queries. Upon receiving the challenge message pair $\{\mathbf{m}_0, \mathbf{m}_1\}$ from $\mathcal{A}$, $\mathcal{S}$ proceeds as follows:

1. **Prepare Ciphertexts:** For each bit $b \in \{0, 1\}$, user $i \in [q_u]$, and level $\ell \in [L]$, $\mathcal{S}$ generates q_r fresh ciphertexts $\text{ct}_{b,i,\ell}^{(k)} \leftarrow \text{Enc}(\text{pk}_i, \mathbf{m}_b, \ell)$ for $k \in [q_r]$. We denote this set as PreCt .
2. **Pre-Fill Ct:** For every ciphertext in PreCt , $\mathcal{S}$ makes a dummy query to the IND-CCA' oracle: $\mathcal{O}_{\text{ReEnc}}(\text{pk}_i, \text{pk}_{\text{hon}}, \text{ct}_{b,i,\ell}^{(k)}, \ell)$, where pk_{hon} is an arbitrary honest key. This forces the IND-CCA' challenger to add these ciphertexts to its internal set Ct .
3. **Challenge Forwarding:** $\mathcal{S}$ forwards $(\text{pk}_{i^*}, \mathbf{m}_0, \mathbf{m}_1, \ell)$ to the IND-CCA' challenger, receives the challenge ct^* , and returns it to $\mathcal{A}$.

During the simulation, whenever $\mathcal{A}$ queries the IND-HRA encryption oracle for $(\text{pk}_i, \mathbf{m}_b, \ell)$ where $\mathbf{m}_b \in \{\mathbf{m}_0, \mathbf{m}_1\}$, $\mathcal{S}$ does not compute a new encryption. Instead, $\mathcal{S}$ selects an unused ciphertext from PreCt and returns it.

Crucially, if $\mathcal{A}$ subsequently requests to re-encrypt one of these pre-filled ciphertexts to a corrupted user, the IND-CCA' challenger will permit the query. Although these ciphertexts decrypt to $\mathbf{m}_0$ or $\mathbf{m}_1$, they are present in Ct (via Step 2) and are distinct from ct^* . Thus, the modified offline rule is bypassed, and the challenger correctly treats them as independent ciphertexts rather than derivatives of ct^* . Since PreCt is large enough to handle q_r queries, the simulation is perfect. $\square$

5 Generalised Source-Hiding

We redefine Source-Hiding (SH) as a generalised class of properties denoted by $\text{SH}^{(a,b)}$. The superscripts (a, b) indicate the number of re-encryption hops performed on the two challenge ciphertexts being compared. Specifically, $\text{SH}^{(0,1)}$ corresponds to the standard notion where a fresh ciphertext (0 hops) is indistinguishable from a re-encrypted one (1 hop).

5.1 Definitions

Definition 2 (General Source-Hiding). A PRE scheme PRE is $\text{SH}^{(a,b)}$ for $(a, b) \in \{0, 1, \dots, L\}^2$ with $\max\{a, b\} > 0$, if for all PPT adversaries $\mathcal{A}$ and all messages $\mathbf{m} \in \mathcal{M}$:

$$(\text{ct}_0^{(0)}, \text{aux}) \approx_s (\text{ct}_0^{(1)}, \text{aux}), \quad (1)$$

where $\text{ct}_0^{(\beta)}$ is the result of re-encrypting a source ciphertext a times (if $\beta = 0$) or b times (if $\beta = 1$) to reach a common target user.

Experiment Setup:

- *Setup:* $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^L)$. Generate a target key pair $(\text{pk}_0, \text{sk}_0)$.
- *Paths:* Generate two independent chains of key pairs: a “left” chain of length a , $\{(\text{pk}_i^{(0)}, \text{sk}_i^{(0)})\}_{i=1}^a$, and a “right” chain of length b , $\{(\text{pk}_j^{(1)}, \text{sk}_j^{(1)})\}_{j=1}^b$.
- *Re-Encryption Keys:* Generate all necessary re-encryption keys along both chains, ending at pk_0 . Specifically, $\{\text{rk}_{i \rightarrow i+1}^{(0)}\}_{i=1}^{a-1}$ and $\text{rk}_{a \rightarrow 0}^{(0)}$ for the left chain; $\{\text{rk}_{j \rightarrow j+1}^{(1)}\}_{j=1}^{b-1}$ and $\text{rk}_{b \rightarrow 0}^{(1)}$ for the right chain.
- *Ciphertext Generation:*
 - If $a > 0$, encrypt m under $\text{pk}_1^{(0)}$ and sequentially re-encrypt it down the left chain to obtain $\text{ct}_0^{(0)}$ under pk_0 . If $a = 0$, $\text{ct}_0^{(0)} \leftarrow \text{Enc}(\text{pk}_0, m)$.
 - Similarly, generate $\text{ct}_0^{(1)}$ using the right chain (or fresh encryption if $b = 0$).
- *Output:* The adversary receives the target ciphertext $\text{ct}_0^{(\beta)}$ and auxiliary information aux comprising pp , all public keys, the target secret key sk_0 , all intermediate re-encryption keys, and all intermediate ciphertexts.

Remark 2. The definition of $\text{SH}^{(0,0)}$ is meaningless. By symmetry, $\text{SH}^{(a,b)}$ implies $\text{SH}^{(b,a)}$. We provide formal definitions for specific instances ($\text{SH}^{(0,1)}$ and $\text{SH}^{(1,1)}$) in Appendix B.

5.2 Implications

We establish that the standard notion, $\text{SH}^{(0,1)}$, is the strongest, implying all other variants.

Theorem 5.1. $\text{SH}^{(0,1)} \Rightarrow \text{SH}^{(a,b)}$ for any $(a, b) \in [L]^2$.

Proof (Sketch). The proof employs a hybrid argument. The core intuition is that $\text{SH}^{(0,1)}$ allows us to replace any single re-encryption step with a fresh encryption.

Consider the transition from an a -hop ciphertext to a b -hop ciphertext. We construct a sequence of hybrids. In the first phase, we iteratively replace the re-encrypted ciphertext at each hop of the a -chain with a fresh encryption using $\text{SH}^{(1,0)}$ (which holds by symmetry from $\text{SH}^{(0,1)}$). This essentially “dismantles” the a -hop chain into a fresh ciphertext at the target. In the second phase, we use $\text{SH}^{(0,1)}$ to iteratively build up the b -hop chain from the fresh ciphertext, as shown in Fig. 1

For example, to prove $\text{SH}^{(0,1)} \Rightarrow \text{SH}^{(1,1)}$, we introduce an intermediate hybrid with a fresh ciphertext. By the triangle inequality, the advantage is bounded by $\epsilon^{(1,1)} \leq \epsilon^{(1,0)} + \epsilon^{(0,1)} = 2\epsilon^{(0,1)}$. $\square$

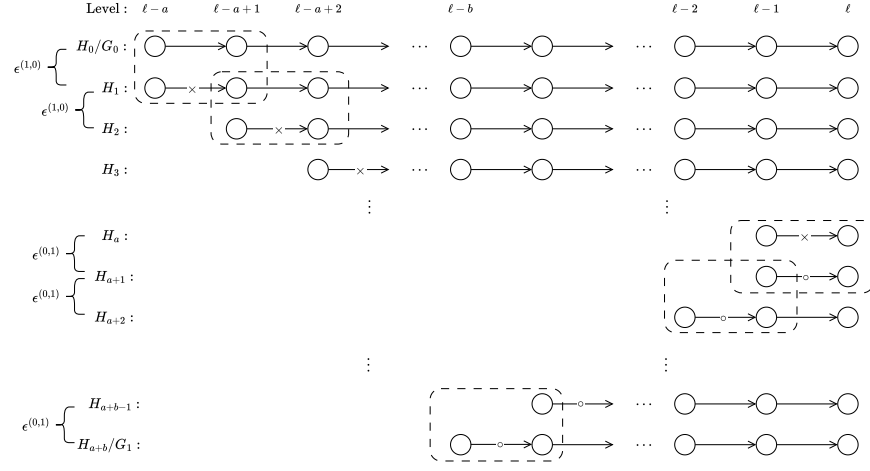


Fig. 1. Hybrid strategy for proving Theorem 5.1. Assuming $a \geq b$, we transform the a -hop chain (G_0) into the b -hop chain (G_1) by incrementally replacing re-encryption steps with fresh encryptions (breaking re-encryption denoted by $\times$), eventually reducing to the trivial case and building back up (new re-encryption denoted by $\circ$).

5.3 Separations

We show that the hierarchy is strict: stronger notions cannot be implied by weaker ones.

Theorem 5.2 ($\text{SH}^{(1,1)} \not\Rightarrow \text{SH}^{(0,1)}$). *There exists a scheme that is $\text{SH}^{(1,1)}$ but not $\text{SH}^{(0,1)}$.*

Proof. We construct a counterexample by appending a “freshness bit”. Let PRE be an IND-CPA secure scheme satisfying $\text{SH}^{(0,1)}$. We construct PRE' as follows:

- $\text{Enc}'(\dots) := \text{Enc}(\dots) \| 0$.
- $\text{ReEnc}'(\dots) := \text{ReEnc}(\dots) \| 1$.
- Dec' ignores the last bit.

PRE' is clearly not $\text{SH}^{(0,1)}$ because the last bit trivially distinguishes fresh (0) from re-encrypted (1) ciphertexts. However, it remains $\text{SH}^{(1,1)}$ because both challenge ciphertexts in the $\text{SH}^{(1,1)}$ game are re-encrypted, thus both end in 1, preserving indistinguishability. $\square$

Theorem 5.3 ($\text{SH}^{(a,a)} \not\Rightarrow \text{SH}^{(a,b)}$ for $a \neq b$). *There exists a scheme that is $\text{SH}^{(a,a)}$ but not $\text{SH}^{(a,b)}$ when $a \neq b$.*

Proof. We construct a counterexample that leaks the parity of the hop count. Let PRE be a $\text{SH}^{(a,b)}$ secure scheme. Construct PRE' such that:

- $\text{Enc}'(\dots) := \text{Enc}(\dots) \parallel 0$.
- $\text{ReEnc}'(\text{rk}, \text{ct} \parallel b, \dots) := \text{ReEnc}(\text{rk}, \text{ct}, \dots) \parallel (b \oplus 1)$.

This scheme appends a bit that flips at each hop. In $\text{SH}^{(a,a)}$, both paths have length a , so the parity bit is identical ($a \bmod 2$), preserving indistinguishability. In $\text{SH}^{(a,b)}$ with $a \neq b$, if a and b have different parity, the last bit trivially distinguishes the ciphertexts. $\square$

6 Reducing IND-CPA to IND-HRA via $\text{SH}^{(a,a)}$

Theorem 6.1. *If a PRE scheme is IND-CPA secure and satisfies $\text{SH}^{(a,a)}$, then it is IND-HRA secure.*

Proof (Sketch). The goal is to construct a simulator $\mathcal{S}$ that uses an IND-CPA challenger to win the IND-HRA game. The primary challenge lies in answering $\mathcal{A}$'s queries to the re-encryption oracle $\mathcal{O}_{\text{ReEnc}}(\text{pk}_i, \text{pk}_j, \text{ct}_i)$, particularly when pk_i is honest and pk_j is corrupted. The IND-CPA challenger does not provide a re-encryption oracle, nor does it allow revealing the re-encryption key $\text{rk}_{i \rightarrow j}$ (since pk_i is honest).

However, assuming the scheme satisfies $\text{SH}^{(a,a)}$, $\mathcal{S}$ can simulate the output ciphertext by constructing an alternative re-encryption path. Instead of computing the real re-encryption $\text{ct}_i \rightarrow \text{ct}_j$ (which requires the unknown $\text{rk}_{i \rightarrow j}$), $\mathcal{S}$ generates a dummy path of length a that originates from a fresh ciphertext but eventually leads to the target corrupted user pk_j at the correct level ℓ .

As illustrated in Fig. 2, if the original ciphertext ct_i is at level $\ell - 1$, $\mathcal{S}$ can generate a fresh ciphertext and re-encrypt it along a simulated path of length a to produce a ciphertext for pk_j . The $\text{SH}^{(a,a)}$ property guarantees that this simulated output is indistinguishable from the result of the real re-encryption.

Let q be the number of re-encryption queries and L the maximum re-encryption level. The reduction incurs a security loss of $O(qL)$ due to the hybrid arguments required for each query simulation. $\square$

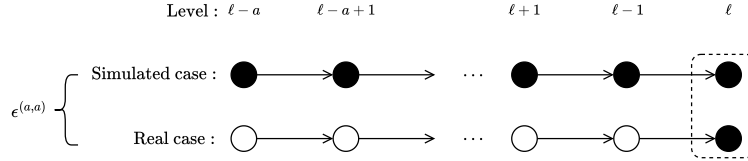


Fig. 2. Simulation strategy for re-encryption queries from an honest user (hollow circle) to a corrupted user (solid circle). The simulator replaces the real re-encryption path (top) with a simulated path (bottom) that results in an indistinguishable target ciphertext at level ℓ , leveraging the $\text{SH}^{(a,a)}$ property.

7 Re-Encryption Key Unlearnability

The security models for Proxy Re-Encryption (IND-HRA and IND-CCA) are built on the principle that re-encryption operations should not compromise the confidentiality of unrelated ciphertexts. Implicit in this is the assumption that the re-encryption process should not leak the underlying re-encryption key itself.

We formalise this intuition by introducing a new security notion: *Re-Encryption Key Unlearnability* (RKUL). This property captures the difficulty of recovering a functional re-encryption key given access to input-output ciphertext pairs. Unlike master secret security [1], which protects the delegator's secret key from collusion, RKUL focuses specifically on the secrecy of the re-encryption key exposed through legitimate re-encryption queries.

RKUL serves as a necessary condition for stronger security notions. We prove that both IND-CCA and IND-HRA imply RKUL, establishing it as a fundamental baseline. Conversely, we show that IND-CPA does not imply RKUL, meaning a scheme can be IND-CPA secure while trivially leaking re-encryption keys. Thus, RKUL provides a meaningful security benchmark that sits strictly between IND-CPA and IND-HRA.

7.1 Definition of RKUL

The goal of the RKUL adversary is to produce a valid re-encryption key for a target pair of users (honest delegator to corrupted delegatee) after observing legitimate re-encryption traffic. We do not require the adversary to output the exact bitwise value of the original key, as probabilistic generation algorithms may produce multiple valid keys. Instead, the adversary wins if it outputs *any* key that functionally performs the correct re-encryption transformation.

Definition 3 (RKUL Game). Let λ be the security parameter and $\mathcal{A}$ a PPT adversary. The RKUL game proceeds as follows:

Phase 1 (Setup & Keys): The challenger generates public parameters pp . $\mathcal{A}$ queries $\mathcal{O}_{\text{KeyGen}}$ to populate sets of honest (Hon) and corrupted (Cor) users.

Phase 2 (Oracles): For all pairs (i, j) , the challenger computes $\text{rk}_{i \rightarrow j}$. $\mathcal{A}$ has access to:

- $\mathcal{O}_{\text{ReKeyGen}}(\text{pk}_i, \text{pk}_j)$: Returns $\text{rk}_{i \rightarrow j}$. Rejects if $\text{pk}_i \in \text{Hon} \wedge \text{pk}_j \in \text{Cor}$.
- $\mathcal{O}_{\text{Enc}}(\text{pk}_i, m, \ell)$: Returns $\text{Enc}(\text{pk}_i, m, \ell)$.
- $\mathcal{O}_{\text{ReEnc}}(\text{pk}_i, \text{pk}_j, \text{ct}_i)$: Computes $(\text{ct}_j, \ell + 1) \leftarrow \text{ReEnc}(\text{rk}_{i \rightarrow j}, \text{ct}_i, \ell)$. Rejects if $\text{pk}_i \in \text{Hon} \wedge \text{pk}_j \in \text{Cor}$ unless the input ct_i was generated by $\mathcal{O}_{\text{Enc}}$ (modelling honest re-encryption).

Phase 3 (Decision): $\mathcal{A}$ outputs a target tuple $(\text{pk}_i, \text{pk}_j, \text{rk}')$. The challenger verifies if:

1. $\text{pk}_i \in \text{Hon}$ and $\text{pk}_j \in \text{Cor}$.
2. $\mathcal{A}$ has not queried $\mathcal{O}_{\text{ReKeyGen}}(\text{pk}_i, \text{pk}_j)$.

3. rk' is functionally correct. The challenger samples a random test message $\mathbf{m} \leftarrow \mathcal{M}$, encrypts it to $(\text{ct}_{\text{test}}, \ell) \leftarrow \text{Enc}(\text{pk}_i, \mathbf{m}, \ell)$, and checks if:

$$\text{Dec}(\text{sk}_j, \text{ReEnc}(\text{rk}', \text{ct}_{\text{test}}, \ell), \ell + 1) = \mathbf{m}.$$

If all conditions hold, $\mathcal{A}$ wins.

Remark 3. To ensure compatibility with IND-HRA, the re-encryption oracle in our RKUL definition is restricted to honest inputs (i.e., ciphertexts generated by $\mathcal{O}_{\text{Enc}}$). A stronger definition allowing arbitrary ciphertext inputs would be implied by IND-CCA but not necessarily by IND-HRA.

7.2 Implications

Theorem 7.1 (IND-CCA $\Rightarrow$ RKUL). *If a PRE scheme is IND-CCA secure, it satisfies RKUL.*

Proof. We construct an IND-CCA adversary $\mathcal{S}$ using an RKUL adversary $\mathcal{A}$. $\mathcal{S}$ simulates the RKUL game for $\mathcal{A}$ by forwarding all key generation and oracle queries to the IND-CCA challenger. Since the RKUL re-encryption oracle is a subset of the IND-CCA re-encryption oracle (restricted to honest inputs), $\mathcal{S}$ can perfectly answer all queries.

Eventually, $\mathcal{A}$ outputs a candidate key tuple $(\text{pk}_i, \text{pk}_j, \text{rk}')$. $\mathcal{S}$ must determine if rk' is valid to leverage it for the IND-CCA challenge.

1. $\mathcal{S}$ selects two distinct messages $\mathbf{m}_0, \mathbf{m}_1$ and requests the IND-CCA challenge on the honest key pk_i . The challenger returns (ct^*, ℓ) (encryption of $\mathbf{m}_b$).
2. Since pk_j is corrupted, $\mathcal{S}$ knows sk_j . $\mathcal{S}$ computes the re-encryption of the challenge using $\mathcal{A}$'s guessed key: $(\text{ct}', \ell + 1) \leftarrow \text{ReEnc}(\text{rk}', \text{ct}^*, \ell)$.
3. $\mathcal{S}$ decrypts $\mathbf{m}' \leftarrow \text{Dec}(\text{sk}_j, \text{ct}', \ell + 1)$.
4. If $\mathbf{m}' = \mathbf{m}_0$, $\mathcal{S}$ guesses $b = 0$; if $\mathbf{m}' = \mathbf{m}_1$, $\mathcal{S}$ guesses $b = 1$.

If $\mathcal{A}$ wins the RKUL game, rk' is a valid key, meaning it correctly re-encrypts ct^* to a ciphertext decryptable by sk_j . Thus, $\mathcal{S}$ will decrypt $\mathbf{m}_b$ correctly and win the IND-CCA game with the same advantage as $\mathcal{A}$. $\square$

Corollary 7.1 (IND-HRA $\Rightarrow$ RKUL). *If a PRE scheme is IND-HRA secure, it satisfies RKUL.*

Proof. The proof is identical to Theorem 7.1, as the IND-HRA challenger also provides the necessary encryption and re-encryption oracles to support the simulation and the validity check of rk' . $\square$

7.3 Separations

We demonstrate that IND-CPA security is insufficient to guarantee re-encryption key secrecy.

Lemma 7.1 (IND-CPA $\not\Rightarrow$ RKUL). *There exists a PRE scheme that is IND-CPA secure but not RKUL secure.*

Proof. Consider any IND-CPA secure PRE scheme Π . We construct a modified scheme Π' that appends the re-encryption key to the ciphertext during re-encryption:

$$\text{ReEnc}'(\text{rk}, \text{ct}) := \text{ReEnc}(\text{rk}, \text{ct}) \parallel \text{rk}$$

The decryption algorithm Dec' simply ignores the appended key. As shown by Cohen [3], appending auxiliary information during re-encryption does not violate IND-CPA security (since the adversary cannot query re-encryptions in the CPA game).

However, an RKUL adversary can query $\mathcal{O}_{\text{ReEnc}}$ on an honest ciphertext and trivially extract rk from the output. The adversary then outputs this key and wins with probability 1. Thus, Π' is IND-CPA secure but trivially insecure under RKUL. $\square$

8 Applying RKUL: Analysis of Existing Schemes

We apply the RKUL framework to evaluate the security of several recent PRE schemes. We identify a recurring design flaw in lattice-based constructions where the re-encryption mechanism is deterministic and linear. This vulnerability allows an adversary to efficiently recover the re-encryption key, violating RKUL and, by extension, the claimed IND-HRA security.

8.1 Case Study: Attribute-Based PRE

Susilo et al. [12] proposed an attribute-based PRE scheme (**abPRE**) based on Learning With Errors (LWE), claiming IND-HRA security. Unlike trapdoor-based constructions [6, 18] that sample short matrices, this scheme encrypts the delegatee's secret key to generate the re-encryption key.

Scheme Overview. We omit the full details of the LWE trapdoor generation and sampling algorithms (see [10] for standard definitions) and focus on the re-encryption mechanism. The re-encryption key $\text{rk}_{f \rightarrow g}$ transforms a ciphertext under policy f to one under policy g .

- **Input:** A ciphertext ct parsed as $(\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_\ell, \mathbf{c}_{out})$.
- **Homomorphic Evaluation:** The proxy evaluates the policy f to obtain $\mathbf{c}_f$ and forms a vector $\mathbf{v} = [\text{BD}((\mathbf{c}'_f)^\top) \parallel \mathbf{c}_{out}^\top]$, where $\mathbf{c}'_f = [\mathbf{c}_{in} \parallel \mathbf{c}_f]$ and BD is the bit-decomposition function.
- **Re-Encryption:** The new ciphertext $\bar{\text{ct}}$ is computed via a linear transformation:

$$\bar{\text{ct}}^\top = \mathbf{v} \cdot \text{rk}_{f \rightarrow g}$$

Crucially, this transformation is *deterministic* and *linear* with respect to the unknown key $\text{rk}_{f \rightarrow g}$.

8.2 The RKUL Attack

We demonstrate that an RKUL adversary can recover $\text{rk}_{f \rightarrow g}$ by collecting sufficiently many input-output pairs.

Attack Strategy.

1. **Query Encryption:** The adversary selects a challenge policy f and queries the encryption oracle for $N = 2mk + m$ random messages, receiving input ciphertexts $\{\text{ct}_i\}_{i=1}^N$.
2. **Query Re-Encryption:** The adversary queries the re-encryption oracle for each ct_i , receiving output ciphertexts $\{\bar{\text{ct}}_i\}_{i=1}^N$.
3. **Linear System Construction:** For each pair $(\text{ct}_i, \bar{\text{ct}}_i)$, the adversary computes the intermediate vector $\mathbf{v}_i$ (which depends only on public parameters and ct_i). These vectors form the rows of a matrix $\mathbf{C} \in \mathbb{Z}_q^{N \times N}$, and the output ciphertexts form a matrix $\mathbf{D}$. The relationship is:

$$\mathbf{D} = \mathbf{C} \cdot \text{rk}_{f \rightarrow g}$$

4. **Key Recovery:** If $\mathbf{C}$ is invertible, the adversary computes $\text{rk}_{f \rightarrow g} = \mathbf{C}^{-1}\mathbf{D}$.

Success Probability. The attack succeeds if $\mathbf{C}$ is full rank. The entries of $\mathbf{v}_i$ come from LWE samples, which are computationally indistinguishable from uniform. Assuming the ciphertexts behave like random vectors, the probability that a random square matrix over $\mathbb{Z}_q$ is invertible is high. For example, over $\mathbb{Z}_2$, the probability is approximately 0.289 [5]. Thus, the adversary recovers the key with non-negligible probability. By Corollary 7.1, the scheme fails IND-HRA security.

Root Cause and Mitigation. The vulnerability stems from the unmasked linear structure of the re-encryption. While determinism itself is not fatal (as seen in pairing-based schemes [1] and analysis in Appendix A), deterministic linearity in lattice schemes allows simple Gaussian elimination attacks. To fix this, re-encryption must include *noise flooding* or *re-randomisation* to mask the linear relationship, a technique successfully applied in [17] and [7].

8.3 Widespread Vulnerabilities

This design flaw—unprotected linear re-encryption—appears in numerous other works, invalidating their security claims.

Lattice-Based Schemes.

- **Tag-Based IND-CCA [6]:** Despite aiming for stronger IND-CCA security, this scheme uses a vulnerable linear transformation for re-encryption, failing RKUL.

- **Attribute-Based Extensions [13, 14]:** These works extend Susilo et al.’s construction but inherit the deterministic re-encryption flaw, leaving them vulnerable to the same linear algebra attack.
- **Threshold and Time-Based PRE [15, 16]:** These schemes split the key (threshold) or constrain it (time), but the underlying re-encryption operation remains linear. An adversary can recover the individual key shares or the time-constrained key, bypassing the intended access controls.

Flawed Security Proofs. A common pattern in these papers is a security proof that reduces the hardness of LWE to the indistinguishability of the challenge ciphertext, *assuming* the re-encryption key is unknown. However, because the adversary can trivially learn the re-encryption key (via our attack) using legal oracle queries, they can decrypt the challenge ciphertext directly. The reduction fails because it does not account for the information leakage from the re-encryption oracle.

Our attack highlights the importance of hiding the re-encryption key during re-encryption, not just the plaintext.

References

1. Ateniese, G., Fu, K., Green, M., Hohenberger, S.: Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Trans. Inf. Syst. Secur.* **9**(1), 1–30 (Feb 2006). <https://doi.org/10.1145/1127345.1127346>, <https://dl.acm.org/doi/10.1145/1127345.1127346>
2. Canetti, R., Hohenberger, S.: Chosen-ciphertext secure proxy re-encryption. In: *Proceedings of the 14th ACM Conference on Computer and Communications Security*. pp. 185–194. CCS ’07, Association for Computing Machinery, New York, NY, USA (Oct 2007). <https://doi.org/10.1145/1315245.1315269>, <https://dl.acm.org/doi/10.1145/1315245.1315269>
3. Cohen, A.: What about Bob? The Inadequacy of CPA Security for Proxy Re-encryption (2017), <https://eprint.iacr.org/2017/785>
4. Cohen, A.: What About Bob? The Inadequacy of CPA Security for Proxy Re-encryption. In: Lin, D., Sako, K. (eds.) *Public-Key Cryptography – PKC 2019*. pp. 287–316. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-17259-6_10
5. Cooper, C.: On the rank of random matrices. *Random Structures & Algorithms* **16**(2), 209–232 (2000). [https://doi.org/10.1002/\(SICI\)1098-2418\(200003\)16:2<209::AID-RSA6>3.0.CO;2-1](https://doi.org/10.1002/(SICI)1098-2418(200003)16:2<209::AID-RSA6>3.0.CO;2-1), <https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291098-2418%28200003%2916%3A2%3C209%3A%3AAID-RSA6%3E3.0.CO%3B2-1>
6. Fan, X., Liu, F.H.: Proxy Re-Encryption and Re-Signatures from Lattices. In: Deng, R.H., Gauthier-Umaña, V., Ochoa, M., Yung, M. (eds.) *Applied Cryptography and Network Security*. pp. 363–382. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-21568-2_18
7. Fuchsbaauer, G., Kamath, C., Klein, K., Pietrzak, K.: Adaptively Secure Proxy Re-encryption. In: Lin, D., Sako, K. (eds.) *Public-Key Cryptography – PKC 2019*. pp. 317–346. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-17259-6_11

8. Gaddam, S., Luykx, A., Sinha, R., Watson, G.: Reducing HSM Reliance in Payments through Proxy Re-Encryption (2021), <https://eprint.iacr.org/2021/094>
9. Libert, B., Vergnaud, D.: Multi-use unidirectional proxy re-signatures. In: Proceedings of the 15th ACM Conference on Computer and Communications Security. pp. 511–520. CCS '08, Association for Computing Machinery, New York, NY, USA (Oct 2008). <https://doi.org/10.1145/1455770.1455835>, <https://dl.acm.org/doi/10.1145/1455770.1455835>
10. Micciancio, D., Peikert, C.: Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology – EUROCRYPT 2012. pp. 700–718. Springer, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-29011-4_41
11. Nuñez, D., Agudo, I., Lopez, J.: A Parametric Family of Attack Models for Proxy Re-encryption. In: 2015 IEEE 28th Computer Security Foundations Symposium. pp. 290–301 (Jul 2015). <https://doi.org/10.1109/CSF.2015.27>, <https://ieeexplore.ieee.org/document/7243740>
12. Susilo, W., Dutta, P., Duong, D.H., Roy, P.S.: Lattice-Based HRA-secure Attribute-Based Proxy Re-Encryption in Standard Model. In: Bertino, E., Shulman, H., Waidner, M. (eds.) Computer Security – ESORICS 2021. pp. 169–191. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-88428-4_9
13. Yao, L., Weng, J., Wu, P., Li, X., Liu, Y., Lai, J., Yang, G., Deng, R.H.: Conditional Attribute-Based Proxy Re-Encryption: Definitions and Constructions from LWE (2022), <https://eprint.iacr.org/2022/637>
14. Zhang, Y., Wang, Y., Wang, M.: Improved AB-CPREs with HRA Security. In: 2023 International Conference on Data Security and Privacy Protection (DSPP). pp. 223–231 (Oct 2023). <https://doi.org/10.1109/DSPP58763.2023.10404893>, <https://ieeexplore.ieee.org/abstract/document/10404893>
15. Zhao, F., Weng, J., Xie, W., Hou, L., Li, M.: Time-based attribute-based proxy re-encryption with decryption key update. Des. Codes Cryptogr. **92**(12), 4099–4129 (Dec 2024). <https://doi.org/10.1007/s10623-024-01467-x>, <https://doi.org/10.1007/s10623-024-01467-x>
16. Zhao, F., Weng, J., Xie, W., Li, M., Weng, J.: HRA-secure attribute-based threshold proxy re-encryption from lattices. Information Sciences **655**, 119900 (Jan 2024). <https://doi.org/10.1016/j.ins.2023.119900>, <https://www.sciencedirect.com/science/article/pii/S0020025523014858>
17. Zhou, Y., Liu, S., Han, S.: Multi-hop Fine-Grained Proxy Re-encryption. In: Tang, Q., Teague, V. (eds.) Public-Key Cryptography – PKC 2024. pp. 161–192. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-57728-4_6
18. Zhou, Y., Liu, S., Han, S., Zhang, H.: Fine-Grained Proxy Re-encryption: Definitions and Constructions from LWE. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023. pp. 199–231. Springer Nature, Singapore (2023). https://doi.org/10.1007/978-981-99-8736-8_7

A HRA-Secure Pairing-Based PRE

We briefly review the pairing-based PRE scheme **pbPRE** proposed by Ateniese et al. [1], let H and G_2 be cyclic groups of prime order $q = \Theta(2^\lambda)$, and let g be

a generator of H . Let $e : H \times H \rightarrow G_2$ be a non-degenerate bilinear map, and define $Z := e(g, g)$. The message space is G_2 .

The scheme consists of the following algorithms:

- $\text{Setup}(1^\lambda, 1^L)$: Output public parameters $\text{pp} = (q, g, H, G_2, e)$.
- $\text{KeyGen}(\text{pp})$: Sample $a \leftarrow \mathbb{Z}_q$ uniformly at random. Output $(\text{sk} = a, \text{pk} = g^a)$.
- $\text{Enc}(\text{pk}, \text{m})$: Sample $k \leftarrow \mathbb{Z}_q$ uniformly at random. Output $\text{ct} = (\text{pk}^k, \text{m}Z^k) = (g^{ak}, \text{m}Z^k)$.
- $\text{ReKeyGen}(\text{sk}_i, \text{pk}_j)$: output $\text{rk}_{i \rightarrow j} = \text{pk}_j^{\frac{1}{\text{sk}_i}} = g^{b/a}$.
- $\text{ReEnc}(\text{rk}_{i \rightarrow j}, \text{ct}_i)$: Let $\text{ct}_i = (\alpha_1, \alpha_2)$. Output

$$\text{ct}_j = (e(\alpha_1, \text{rk}_{i \rightarrow j}), \alpha_2) = (e(g^{ak}, g^{b/a}), \text{m}Z^k) = (Z^{bk}, \text{m}Z^k).$$

- $\text{Dec}(\text{sk}, \text{ct})$: Let $\text{ct} = (\alpha_1, \alpha_2)$. If $\alpha \in G_2$, then output $\alpha_2/\alpha_1^{1/a} = \text{m}Z^k/Z^k = \text{m}$. Otherwise $\alpha_1 \in H$, output $\alpha_2/e(\alpha_1, g)^{1/a} = \text{m}Z^k/e(g^{ak}, g^{1/a}) = \text{m}Z^k/Z^k = \text{m}$.

Cohen [3, Section 5.4] noted that it is difficult to prove this scheme is re-encryption simulatable. Specifically, given a ciphertext $\text{ct}_i = (g^{ak}, \text{m}Z^k)$, public keys $\text{pk}_i = g^a$, $\text{pk}_j = g^b$, secret key $\text{sk}_j = b$, and message m (i.e., the auxiliary input), it is unclear how to simulate the re-encrypted ciphertext $(g^{bk}, \text{m}Z^k)$ using shared randomness k .

However, we observe that this target ciphertext $(g^{bk}, \text{m}Z^k)$ is actually a fresh encryption under pk_j , not the result of re-encryption. The actual re-encrypted ciphertext is $(Z^{bk}, \text{m}Z^k)$, which is simulatable. Specifically, the simulator can compute:

$$\left(\frac{\alpha_2}{\text{m}}\right)^{\text{sk}_j} = \left(\frac{\text{m}Z^k}{\text{m}}\right)^b = Z^{kb}.$$

This observation eliminates the need for re-randomisation in the simulation, making the scheme more efficient while still achieving IND-HRA security.

B Definitions of Source-Hiding

Definition 4 (Source-Hiding^(0,1)). A PRE scheme PRE is $\text{SH}^{(0,1)}$ if for all $\text{m} \in \mathcal{M}$:

$$(\text{ct}_0^{(0)}, \text{aux}) \approx_s (\text{ct}_0^{(1)}, \text{aux}),$$

where $\text{ct}_0^{(0)}, \text{ct}_0^{(1)}$, and aux are sampled according to

$$\begin{aligned}
& \text{pp} \leftarrow \text{Setup}(1^\lambda, 1^L), \\
& (\text{pk}_1^{(1)}, \text{sk}_1^{(1)}), (\text{pk}_0, \text{sk}_0) \leftarrow \text{KeyGen}(\text{pp}), \\
& (\text{ct}_0^{(0)}, \ell) \leftarrow \text{Enc}(\text{pk}_0, \text{m}, \ell), \\
& (\text{ct}_1^{(1)}, \ell - 1) \leftarrow \text{Enc}(\text{pk}_1^{(1)}, \text{m}, \ell - 1), \\
& \text{rk}_{1 \rightarrow 0}^{(1)} \leftarrow \text{ReKeyGen}(\text{sk}_1^{(1)}, \text{pk}_0), \\
& (\text{ct}_0^{(1)}, \ell) \leftarrow \text{ReEnc}(\text{rk}_{1 \rightarrow 0}^{(1)}, \text{ct}_1^{(1)}, \ell - 1), \\
& \text{aux} := (\text{pp}, \text{pk}_1^{(1)}, \text{pk}_0, \text{sk}_0, \text{ct}_1^{(1)}, \ell, \text{m}).
\end{aligned}$$

Definition 5 (Source-Hiding^(1,1)). A PRE scheme PRE is $SH^{(1,1)}$ if for all $\text{m} \in \mathcal{M}$:

$$(\text{ct}_0^{(0)}, \text{aux}) \approx_s (\text{ct}_0^{(1)}, \text{aux}),$$

where $\text{ct}_0^{(0)}, \text{ct}_0^{(1)}$, and aux are sampled according to

$$\begin{aligned}
& \text{pp} \leftarrow \text{Setup}(1^\lambda, 1^L), \\
& (\text{pk}_1^{(0)}, \text{sk}_1^{(0)}), (\text{pk}_1^{(1)}, \text{sk}_1^{(1)}), (\text{pk}_0, \text{sk}_0) \leftarrow \text{KeyGen}(\text{pp}), \\
& (\text{ct}_1^{(0)}, \ell - 1) \leftarrow \text{Enc}(\text{pk}_1^{(0)}, \text{m}, \ell - 1), \\
& (\text{ct}_1^{(1)}, \ell - 1) \leftarrow \text{Enc}(\text{pk}_1^{(1)}, \text{m}, \ell - 1), \\
& \text{rk}_{1 \rightarrow 0}^{(0)} \leftarrow \text{ReKeyGen}(\text{sk}_1^{(0)}, \text{pk}_0), \\
& \text{rk}_{1 \rightarrow 0}^{(1)} \leftarrow \text{ReKeyGen}(\text{sk}_1^{(1)}, \text{pk}_0), \\
& (\text{ct}_0^{(0)}, \ell) \leftarrow \text{ReEnc}(\text{rk}_{1 \rightarrow 0}^{(0)}, \text{ct}_1^{(0)}, \ell - 1), \\
& (\text{ct}_0^{(1)}, \ell) \leftarrow \text{ReEnc}(\text{rk}_{1 \rightarrow 0}^{(1)}, \text{ct}_1^{(1)}, \ell - 1), \\
& \text{aux} := (\text{pp}, \text{pk}_1^{(0)}, \text{pk}_1^{(1)}, \text{pk}_0, \text{sk}_0, \text{ct}_1^{(0)}, \text{ct}_1^{(1)}, \ell, \text{m}).
\end{aligned}$$